

Deep Learning – Homework#12-1

Operations on word vectors

Welcome to your first assignment of this week!

Because word embeddings are very computationally expensive to train, most ML practitioners will load a pre-trained set of embeddings.

After this assignment you will be able to:

- Load pre-trained word vectors, and measure similarity using cosine similarity
- Use word embeddings to solve word analogy problems such as Man is to Woman as King is to _____.
- Modify word embeddings to reduce their gender bias

Let's get started! Run the following cell to load the packages you will need.

```
import numpy as np
from w2v_utils import *
```

Next, let's load the word vectors. For this assignment, we will use 50-dimensional GloVe vectors to represent words. Run the following cell to load the `word_to_vec_map`.

```
words, word_to_vec_map = read_glove_vecs('data/glove.6B.50d.txt')
```

You've loaded:

- `words`: set of words in the vocabulary.
- `word_to_vec_map`: dictionary mapping words to their GloVe vector representation.

You've seen that one-hot vectors do not do a good job capturing what words are similar. GloVe vectors provide much more useful information about the meaning of individual words. Let's now see how you can use GloVe vectors to decide how similar two words are.

1 Cosine similarity

To measure how similar two words are, we need a way to measure the degree of similarity between two embedding vectors for the two words. Given two vectors u and v , cosine similarity is defined as follows:

$$\text{CosineSimilarity}(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2} = \cos(\theta) \quad (1)$$

where $u \cdot v$ is the dot product (or inner product) of two vectors, $\|u\|_2$ is the norm (or length) of the vector u , and θ is the angle between u and v . This similarity depends on the angle between u and v . If u and v are very similar, their cosine similarity will be close to 1; if they are dissimilar, the cosine similarity will take a smaller value.

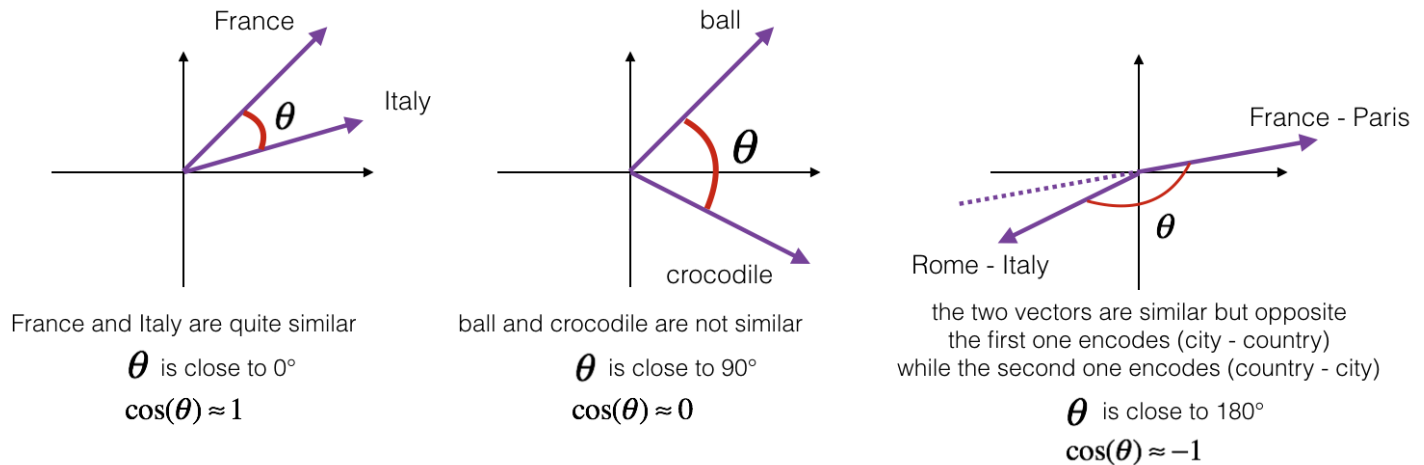


Figure 1: The cosine of the angle between two vectors is a measure of how similar they are.

Exercise: Implement the function `cosine_similarity()` to evaluate similarity between word vectors.

Reminder: The norm of u is defined as $\|u\|_2 = \sqrt{\sum_{i=1}^n u_i^2}$

```
# GRADED FUNCTION: cosine_similarity

def cosine_similarity(u, v):
    """
    Cosine similarity reflects the degree of similarity between u and v

    Arguments:
        u -- a word vector of shape (n,)
        v -- a word vector of shape (n,)

    Returns:
        cosine_similarity -- the cosine similarity between u and v defined by the formula
        above.
    """

    distance = 0.0

    ### START CODE HERE ###
    # Compute the dot product between u and v (~1 line)
    dot = None
    # Compute the L2 norm of u (~1 line)
    norm_u = None

    # Compute the L2 norm of v (~1 line)
```

```

norm_v = None
# Compute the cosine similarity defined by formula (1) (~1 line)
cosine_similarity = None
### END CODE HERE ###

return cosine_similarity

```

```

father = word_to_vec_map["father"]
mother = word_to_vec_map["mother"]
ball = word_to_vec_map["ball"]
crocodile = word_to_vec_map["crocodile"]
france = word_to_vec_map["france"]
italy = word_to_vec_map["italy"]
paris = word_to_vec_map["paris"]
rome = word_to_vec_map["rome"]

print("cosine_similarity(father, mother) = ", cosine_similarity(father, mother))
print("cosine_similarity(ball, crocodile) = ", cosine_similarity(ball, crocodile))
print("cosine_similarity(france - paris, rome - italy) = ", cosine_similarity(france -
    paris, rome - italy))

```

Expected Output:

```

cosine_similarity(father, mother) = 0.8909038442893615
cosine_similarity(ball, crocodile) = 0.2743924626137943
cosine_similarity(france - paris, rome - italy) = -0.6751479308174204

```

After you get the correct expected output, please feel free to modify the inputs and measure the cosine similarity between other pairs of words! Playing around the cosine similarity of other inputs will give you a better sense of how word vectors behave.

2 Word analogy task

In the word analogy task, we complete the sentence “a is to b as c is to ____”. An example is “man is to woman as king is to queen”. In detail, we are trying to find a word **d**, such that the associated word vectors e_a, e_b, e_c, e_d are related in the following manner: $e_b - e_a \approx e_d - e_c$. We will measure the similarity between $e_b - e_a$ and $e_d - e_c$ using cosine similarity.

Exercise: Complete the code below to be able to perform word analogies!

```

# GRADED FUNCTION: complete_analogy

def complete_analogy(word_a, word_b, word_c, word_to_vec_map):
    """
    Performs the word analogy task as explained above: a is to b as c is to ____
    """

```

```

Arguments:
word_a -- a word, string
word_b -- a word, string
word_c -- a word, string
word_to_vec_map -- dictionary that maps words to their corresponding vectors.

Returns:
best_word -- the word such that v_b - v_a is close to v_best_word - v_c, as measured
             by cosine similarity
"""

# convert words to lower case
word_a, word_b, word_c = word_a.lower(), word_b.lower(), word_c.lower()

### START CODE HERE ###
# Get the word embeddings v_a, v_b and v_c (~1-3 lines)
e_a, e_b, e_c = None
### END CODE HERE ###

words = word_to_vec_map.keys()
max_cosine_sim = -100                # Initialize max_cosine_sim to a large negative
                                     # number
best_word = None                     # Initialize best_word with None, it will help keep
                                     # track of the word to output

# loop over the whole word vector set
for w in words:
    # to avoid best_word being one of the input words, pass on them.
    if w in [word_a, word_b, word_c] :
        continue

    ### START CODE HERE ###
    # Compute cosine similarity between the combined vector and the current word (~1
    # line)
    cosine_sim = None

    # If the cosine_sim is more than the max_cosine_sim seen so far,
    # then: set the new max_cosine_sim to the current cosine_sim and the best_word
    # to the current word (~3 lines)
    if None > None:
        max_cosine_sim = None
        best_word = None
    ### END CODE HERE ###

return best_word

```

Run the cell below to test your code, this may take 1-2 minutes.

```

triads_to_try = [('italy', 'italian', 'spain'), ('india', 'delhi', 'japan'), ('man', '
    woman', 'boy'), ('small', 'smaller', 'large')]
for triad in triads_to_try:
    print ('{} -> {} :: {} -> {}'.format(*triad, complete_analogy(*triad, word_to_vec_map)

```

))

Expected Output:

```
italy -> italian :: spain -> spanish
india -> delhi :: japan -> tokyo
man -> woman :: boy -> girl
small -> smaller :: large -> larger
```

Once you get the correct expected output, please feel free to modify the input cells above to test your own analogies. Try to find some other analogy pairs that do work, but also find some where the algorithm doesn't give the right answer: For example, you can try `small->smaller` as `big->?`.

Congratulations!

You've come to the end of this assignment. Here are the main points you should remember:

- Cosine similarity a good way to compare similarity between pairs of word vectors. (Though L2 distance works too.)
- For NLP applications, using a pre-trained set of word vectors from the internet is often a good way to get started.

Even though you have finished the graded portions, we recommend you take a look too at the rest of this assignment.